

Assignment 2

CNN, RNN and LSTM Architectures (15 points)

Checkpoint: 03/15/2026 Sun 11:59pm

Due date: 03/29/2026 Sun 11:59pm

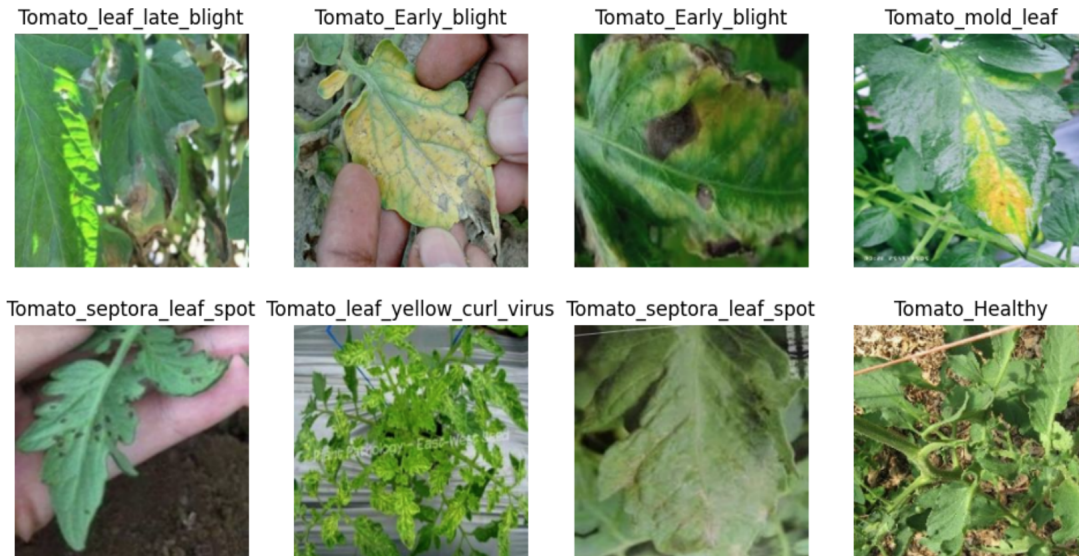
Welcome to Assignment 2! This assignment focuses on developing both theoretical and practical skills related to convolutional neural networks (CNNs), recurrent neural networks (RNNs), and Long Short-Term Memory (LSTM) networks. It consists of two hands-on exercises in which you will analyze network configurations, work with various datasets, and implement, train, and tune different neural networks. There is also a theoretical component covering CNNs and LSTMs.

- The assignment is expected to be fully completed using PyTorch (recommended) or TensorFlow. Mixing is not allowed.
- **You need to change your teammates for assignment 2.**
- At the end of your report, you should have your Team Participation Statement. Submissions without a correctly formatted statement will receive a 0 for the assignment. The team participation statement should have the following:
 - Information on the parts of the assignment that each team member contributed to. A few sentences should be enough (e.g., “Person X contributed to data preprocessing, Person Y contributed to data visualization, and Person Z improved the accuracy with data augmentation.”).
 - A statement by each team member that expresses their explicit agreement with the above-stated distribution of work, i.e., something like “I agree that this statement accurately reflects the distribution of work in our group. - Your Name”.

Part I: From VGG to ResNet [5 pts]

Implement and compare one of the fundamental CNN architectures: VGG-16 (Version C) and ResNet-18 for image classification. Explore advanced techniques to improve model performance and explore the transition from standard deep CNNs to networks with residual connections. The expected accuracy is above 75% for the base model and 80% for the improved model.

Tomato Leaf Disease Dataset



Use the Tomato Leaf Disease Dataset for this part. This dataset contains RGB images of tomato leaves categorized into six classes: five common tomato leaf diseases and one healthy class. Each class has 1200 instances.

Step 1: Data preparation

1. Download the dataset from this link:
<https://www.kaggle.com/datasets/syedhashirali260/tomato-leaf-disease-dataset-6-classes/data>
2. Display a grid of sample images from each class (e.g., a 6×3 grid showing 3 images from each class)
3. Preprocess the dataset.
 - Resize images to appropriate size.
 - Normalize the pixel values to the range $[0, 1]$. E.g. divide by 255.0
 - *[If needed]* Address class imbalance in the target column. Possible solutions: oversampling; undersampling; data augmentation techniques for the minority class; assign higher weights to the minority class and lower weights to the majority class, etc.
 - Convert target variable needs to numerical format. You can use one-hot encoding. However, if you use `torch.nn.CrossEntropyLoss` for your network, it expects class indices (0, 1, 2) directly, not one-hot encoded vectors. Therefore, ensure your labels are integer tensors (e.g., `torch.LongTensor`).

4. Split the dataset into training, testing and validation sets. You can use `train_test_split` from scikit-learn.

Step 2: Implementing VGG

1. Use `torchvision.models` to implement a version of VGG. Refer to the [original VGG paper](#) for more details. Please note that:
 - a. The input size for the original VGG is 224×224 .
 - b. The last layer of the original VGG contains 1000 classes. You can modify it to fit the number of your classes (e.g. 6).
 - c. The expected accuracy is above 70% on test data. You may use pretrained weights or regularization techniques to reach the minimum accuracy.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

2. Train your VGG-16 model on the dataset:
 - a. Experiment with at least two different weight initialization strategies (e.g., Xavier/Glorot, He initialization). Report the impact on training.
 - b. Experiment with at least two different batch sizes (e.g., 32, 64, 128). Report the impact on training speed (time per epoch) and performance.
3. Evaluation and analysis. Include the following metrics:

- Training accuracy, training loss, validation accuracy, validation loss, testing accuracy, and testing loss. Provide your short analysis.
- Plot the training and validation accuracy over time (epochs).
- Plot the training and validation loss over time (epochs).
- Generate a confusion matrix for the test set using `sklearn.metrics.confusion_matrix`. Display it using `seaborn.heatmap`. Analyze the confusion matrix: Which classes are most often confused?
- Calculate and report other evaluation metrics such as precision, recall and F1 score (details). Refer to `sklearn.metrics.precision_recall_fscore_support`. Analyze the results.
- Use [TensorBoard](#) (or a similar tool, e.g. [Wandb](#)) to log the training and validation loss and accuracy over epochs, generate the charts, and attach the SVG images of the charts.
- Display a few misclassified images from the test set along with their predicted and true labels. Try to explain why the model might have made these mistakes.

Step 3: Implementing ResNet

- Use `torchvision.models` to implement a version of ResNet. Find more details about ResNet in [original ResNet paper](#).

layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10 ⁹	3.6×10 ⁹	3.8×10 ⁹	7.6×10 ⁹	11.3×10 ⁹

- Train the ResNet-18 model on the same dataset, using the same data splits, preprocessing, and similar training procedures as in Step 2. The expected accuracy on test data is above 75%. You may use pretrained weights or regularization techniques to reach the minimum accuracy.
- Evaluation and analysis.

- a. Perform the same comprehensive evaluation as in Step 2 (metrics, plots, confusion matrix, precision/recall/F1, TensorBoard, misclassified examples). Provide your analysis.
- b. Create at least two plots that directly compare the performance of ResNet-18 and VGG-16. Use different colors and legends to distinguish the models. Provide your analysis. For example, a plot showing:
 - Training accuracy of both models vs. epoch.
 - Validation accuracy of both models vs. epoch.
 - Training loss of both models vs. epoch.
 - Validation loss of both models vs. epoch.

Step 4: Discussion and conclusion

1. Briefly explain the theoretical concepts behind VGG and ResNet:
 - **VGG:** Discuss the idea of using small convolutional filters (3×3) and stacking multiple layers to increase depth.
 - **ResNet:** Explain the problem of vanishing/exploding gradients in very deep networks and how residual connections address this issue. Explain the concept of “identity mapping.”
2. Discuss how the regularization and optimization techniques you used impacted the performance of both models.
3. Analyze the results, including all the required graphs and metrics (as listed in Step 2, Part 5, and Step 3, Part 5). Focus on the comparison between VGG-16 and ResNet-18. Which model performed better? Why?
4. Summarize your findings, highlighting the key differences between VGG-16 and ResNet-18. Discuss the advantages of residual connections in deep CNNs, based on your experimental results. State which model you would recommend for this task and why.
5. **References.** Include details on all the resources used to complete this part.

Notes:

- You may use the built-in VGG, ResNet, or pretrained models.
- You can partially reuse related implementations that you have completed for other courses with a proper citation, e.g. “Part I, Step 1 is based on the CSE 676 Deep Learning Assignment 1 submission by My Full Name”
- For this part, there is no need to include the report. Submit a single, well-organized Jupyter Notebook containing all code, results, visualizations, and discussion. Make sure the notebook runs without errors. Use markdown cells to clearly separate sections and provide explanations.

- Use `torch.manual_seed()` and `random.seed()` for reproducibility of your results.

Part II: Sentiment analysis using LSTM [5 points]

In this part, we perform a sentiment analysis using an LSTM model. The final (improved) model should achieve a test accuracy of greater than 75%.

DATASETS

Choose ONE dataset from the following options for sentiment analysis:

- [Amazon Product Reviews Dataset](#): reviews of various products sold on Amazon, labeled with sentiment.
- [Yelp Reviews Dataset](#): reviews from the Yelp platform, labeled with sentiment
- [Twitter Airline Sentiment Dataset](#): tweets about airline experiences, labeled with sentiment.
- [Stanford Large Movie Review Dataset](#) movie reviews classified as positive or negative
- [Medical Dialogue Dataset](#): contains conversations between doctors and patients. It has 0.26 million dialogues
- [News Headlines Dataset](#): news headlines labeled with sentiment, covering various news topics and categories.

Step 1: Data exploration and preprocessing

1. Load your chosen dataset and print the main statistics.
2. Print the first 5 rows of the dataset to understand its structure.
3. Provide a brief description (2–3 sentences) of the dataset: What does it represent? What are the sentiment labels? Provide a link to the source.
4. Display descriptive statistics:
 - Number of samples.
 - Class distribution (percentage of positive, negative, etc. reviews).
 - Average review length (in words or characters).
 - Vocabulary size (number of unique words).
5. Handle missing values (if any). Explain your chosen method (e.g., removal, imputation – though removal is usually fine for text data if there are few missing values).
6. Create at least three visualizations to gain insights into the data:

- Histogram of review lengths (both in words and characters).
- Generate word clouds for each sentiment class. Use the [wordcloud library](#) (`pip install wordcloud`). This helps visualize the most frequent words associated with each sentiment.
- Bar chart showing the number of reviews for each sentiment class.
- Any other relevant visualization (e.g., n-gram frequency distributions).

7. Data preparation:

- Tokenize the text data (convert text into sequences of words or subwords). You can use [nltk toolkit](#) or `torchtext.data.utils.get_tokenizer`. Experiment with at least two different tokenization methods (e.g., `nltk.word_tokenize` vs. `get_tokenizer("basic_english")`). Justify your final choice. Compare the vocabulary sizes and processing times.
- Build a vocabulary from the training data only. Map each unique token to an integer index. You can use `torchtext.vocab.build_vocab_from_iterator` or create your own vocabulary mapping. Convert text data to numerical sequences using vocabulary.
- Pad sequences to a uniform length. Choose an appropriate maximum sequence length (`max_length`). You can base this on the distribution of review lengths (e.g., the 95th percentile).
- Use `torch.nn.utils.rnn.pad_sequence` (preferred) or manually pad with zeros.
- Discuss the impact of padding and truncation on model performance. If you truncate very long sequences, you might lose important information. If you pad excessively, you might introduce noise.

8. Split dataset into train, validation, and test sets. Because this is not a time series task, you can use `train_test_split`

Step 2: Baseline LSTM Model

1. Build an LSTM model with the following characteristics:

- Embedding layer to represent words as dense vectors. Experiment with different embedding dimensions and explain your choice.
- Include at least three LSTM layers (`torch.nn.LSTM`). Experiment with the number of hidden units in each layer.
- Use dropout (`torch.nn.Dropout`) after each LSTM layer for regularization.
- Include one or more fully connected layers (`torch.nn.Linear`) to map the LSTM output to the number of sentiment classes.
- Use the output layer with an appropriate activation function (e.g., softmax for multi-class classification, sigmoid for binary classification).

- Print model summary using `torchinfo.summary`
2. Train your model:
 - Choose an appropriate loss function (e.g., cross-entropy) and optimizer (e.g., Adam).
 - Train your model and monitor its performance on the training and validation sets.
 - Tune hyperparameters (learning rate, batch size, number of epochs, hidden units, dropout rate) using the validation set. Describe your tuning strategy (e.g., grid search, random search, manual tuning) and document the different combinations you tried.
 - Plot the training and validation loss and accuracy curves over epochs. Analyze the plots for signs of overfitting or underfitting. Adjust your model or hyperparameters if needed.
 3. Evaluation and analysis:
 - Evaluate your best model on the test set. Report the accuracy, loss, and any other relevant metrics (precision, recall, F1-score, etc.).
 - Generate a confusion matrix (using `sklearn.metrics.confusion_matrix` and `seaborn.heatmap`). Analyze the confusion matrix.

Step 3: Improved LSTM Model

1. Improve your baseline LSTM model by incorporating one or more of the following techniques:
 - Bidirectional LSTM. Use `bidirectional=True` in your LSTM layer.
 - Stacked LSTM. You already have at least three layers; you can add more if desired.
 - GRU (Gated Recurrent Unit): Replace LSTM layers with `torch.nn.GRU`.
 - Attention Mechanism. Implement an attention mechanism. This is more advanced.
 - Pre-trained Word Embeddings. Initialize your embedding layer with pre-trained word embeddings (e.g., GloVe, Word2Vec, FastText). You can download pre-trained embeddings from various sources (e.g., the GloVe website). Use `torchtext.vocab.Vectors` to load pre-trained embeddings. This is highly recommended.
2. Create a new class for your improved model.
3. Follow the same training and evaluation procedures as in step 2 (baseline LSTM model). Tune hyperparameters and analyze the performance on the training, validation, and test sets.
4. Directly compare the performance of your improved model to the baseline model. Use the same metrics and create plots that show both models' performance (e.g., training/validation curves on the same plot). Discuss the reasons for any observed improvements (or lack thereof). Analyze the impact of the specific techniques you used.

Step 4: Discussion

1. Briefly describe the dataset and its key characteristics.
2. Describe both your baseline and improved LSTM architectures in detail.
3. Discuss the results, comparing the performance of the baseline and improved models. Refer to specific metrics and visualizations.
4. Discuss the strengths and limitations of using recurrent neural models (specifically LSTMs) for sentiment analysis. Consider aspects like:
 - Handling long sequences.
 - Computational cost.
 - Interpretability (compared to simpler models like logistic regression).
 - Sensitivity to hyperparameters.
5. **References.** Include details on all the resources used to complete this part.

Part III: CNN & LSTM Theoretical Part [5 points]

This part focuses on the theoretical analysis of CNNs and LSTMs through problem-solving. Provide clear, concise solutions that demonstrate your complete work and reasoning. Answers may be handwritten (legibly) or prepared using MS Word or $\text{\LaTeX}$.

Part III.I: CNN [2 points]

You are working on a computer vision project using a CNN to classify images of human faces expressing different emotions. This is a multi-class classification task with 5 output classes: happiness, sadness, anger, surprise, and neutral.

CNN Architecture:

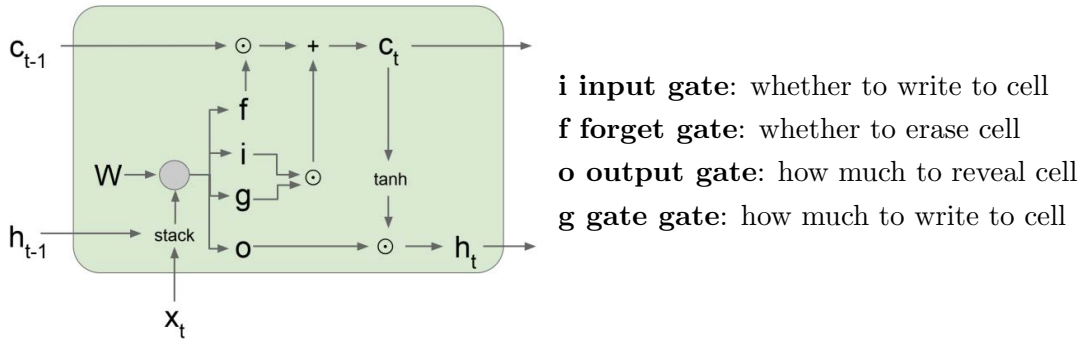
- Input: 32×28 RGB images.
- First Layer: Convolutional layer with 10 filters, each of size 5×5 , zero padding, and a stride of 1.
- Output: Predicted probabilities for each category (5 classes), summing to 1.

TASKS:

1. Calculate the output size (dimensions) after the first convolutional layer. Show your calculations.
2. Determine the total number of learnable parameters in the first convolutional layer. Show your calculations.
3. Recalculate the output size after the first convolutional layer if a padding of 1 was used instead of zero padding. Show your calculations.
4. Calculate the number of parameters in the first layer if the input images were grayscale (single channel) instead of RGB. Show your calculations.
5. Given the task and the requirement of probabilistic outputs, which activation function is most suitable for the output layer? Explain your choice and why other common activation functions are less appropriate in this scenario.
6. Prove that the activation function you chose in Task 5 is invariant to constant shifts in the input values. Mathematically demonstrate that adding a constant value to all input values will not change the resulting output probabilities. Explain the significance of shift invariance in this context.

Part III.II: LSTM Derivation [3 points]

Consider the following standard LSTM structure



The definitions are provided as follows:

$$\begin{pmatrix} \mathbf{i}_t \\ \mathbf{f}_t \\ \mathbf{o}_t \\ \mathbf{g}_t \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} \left[\begin{pmatrix} \mathbf{W}_1 \\ \mathbf{W}_2 \\ \mathbf{W}_3 \\ \mathbf{W}_4 \end{pmatrix} \begin{pmatrix} \mathbf{h}_{t-1} \\ \mathbf{x}_t \end{pmatrix} \right]$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \mathbf{g}_t$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

where:

- σ denotes the sigmoid activation function
- $\mathbf{W}_1, \mathbf{W}_2, \mathbf{W}_3, \mathbf{W}_4$ represent weight matrices
- $\mathbf{h}_{t-1}$ represents the hidden state at previous time step
- $\mathbf{x}_t$ represents the current input.

TASKS:

Assume $\mathcal{L}$ is the loss of an LSTM model at time step t . Derive the gradient formulas for the following partial derivatives, expressing them in terms of the LSTM variables:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{i}_t}, \frac{\partial \mathcal{L}}{\partial \mathbf{f}_t}, \frac{\partial \mathcal{L}}{\partial \mathbf{o}_t}, \frac{\partial \mathcal{L}}{\partial \mathbf{g}_t}, \frac{\partial \mathcal{L}}{\partial \mathbf{c}_t}, \frac{\partial \mathcal{L}}{\partial \mathbf{h}_t}$$

Submission of theoretical part (Part III):

Your final PDF for Part III should include clear, step-by-step derivations for all requested calculations. Justify each step and clearly mark your final answers.

You may provide the answer using handwritten notes, typed in the MS Word or L^AT_EX. For the final submission, combine all your answers into a single PDF named:

a2_part_III_TEAMMATE1_TEAMMATE2.pdf

e.g. a2_part_III_mgao8_pouyakar.pdf

Bonus points, implementing ResNeXt [2 points]

Implement [ResNeXt](#) architecture. It is expected that your accuracy will be higher than ResNet. Compare the results with your VGG and ResNet implementation.

STEPS:

1. Implement the [ResNeXt](#) architecture, using `torchvision.models`.
2. Train and evaluate your ResNeXt model on the same dataset used in Part I.
3. Compare the performance of your ResNeXt model against your previous ResNet and VGG models. Provide a detailed analysis of the results, including:
 - a. A table summarizing the performance metrics (accuracy, loss, etc.) for all three models.

- b. Discussion of the observed differences in performance. Explain why ResNeXt might be outperforming ResNet and VGG or having worse results. Consider factors like cardinality, grouped convolutions, and the overall architecture.
 - c. Analysis of any challenges encountered during the implementation or training process.
4. **References.** Include details on all the resources used to complete this part.

SUBMISSION:

Submit your ResNeXt implementation as a Jupyter Notebook file named:

`a2_bonus_resnext_TEAMMATE1_TEAMMATE2.ipynb`

Ensure your notebook includes clear comments explaining your code and analysis.

ASSIGNMENT STEPS

1. Submit checkpoint: Part I (03/15/2026 Sun)

- Complete Part I: CNN of the assignment.
- Include all the references at the end of each part.
- Your Jupyter notebook should be saved with the outputs.
- Combine all files in a single zip folder that will be submitted on UBlearns, named as `assignment2_checkpoint_YOUR.UBIT.zip`
- Submit to UBlearns > Assignments
- Suggested file structure:
`assignment2_checkpoint_TEAMMATE1_TEAMMATE2.zip`
 - `a2_part_1_TEAMMATE1_TEAMMATE2.ipynb`

2. Submit final results (03/29/2026 Sun)

- Fully complete all parts of the assignment.
- Add all your assignment files in a zip folder including ipynb files for Part I, Part II, Part III & Bonus part (optional).
- Name zip folder with all the files as `assignment2_final_TEAMMATE1_TEAMMATE2.zip`, e.g. `assignment2_final_mgao8_pouyakar.zip`
- Submit to UBlearns > Assignments
- Your Jupyter notebook should be saved with the outputs.

- Include all the references at the end of each part that have been used to complete that part.
- You can make unlimited number of submissions and only the latest will be evaluated
- Suggested file structure (bonus part is optional):
assignment2_final_TEAMMATE1_TEAMMATE2.zip
 - a2_part_1_TEAMMATE1_TEAMMATE2.ipynb
 - a2_part_2_TEAMMATE1_TEAMMATE2.ipynb
 - a2_part_3_TEAMMATE1_TEAMMATE2.pdf
 - a2_bonus_resnext_TEAMMATE1_TEAMMATE2.ipynb

Notes:

- Ensure your code is well-organized and includes comments explaining key functions and attributes. After running the Jupyter Notebooks, all results and plots used in your report should be generated and clearly displayed.
- You can submit multiple files, but they all need to be labeled with a clear name.
- Recheck the submitted files, e.g. download and open them, once submitted and verify that they open correctly
- The zip folder should include all relevant files, clearly named as specified.

Academic Integrity

The standing policy of the Department is that all students involved in any academic integrity violation (e.g., plagiarism in any way, shape, or form) will receive an F grade for the course. The catalog describes plagiarism as “Copying or receiving material from any source and submitting that material as one’s own, without acknowledging and citing the particular debts to the source, or in any other manner representing the work of another as one’s own.”. Refer to the [Office of Academic Integrity](#) for more details.

Important Information

This assignment can be done individually or in a team of up to two students. The grading rubrics are the same for a team of any size.

- No collaboration, cheating, and plagiarism is allowed in assignments, quizzes, the midterms or final project.

- All the submissions will be checked using MOSS as well as other tools. MOSS is based on the submitted works for the past semesters as well the current submissions. We can see all the sources, so you don't need to worry if there is a high similarity with your Checkpoint submission.
- The submissions should include all the references. Kindly note that referencing the source does not mean you can copy/paste it fully and submit as your original work. Updating the hyperparameters or modifying the existing code is a subject to plagiarism. Your work has to be original. If you have any doubts, send a private post on piazza to confirm.
- All parties involved in any suspicious cases will be officially reported using the Academic Dishonesty Report form. Please refer to the [Academic Integrity Policy](#) for more details.

Late Days Policy

You can use up to 5 late days throughout the course that can be applied to any assignment-related due dates. You do not have to inform the instructor, as the late submission will be tracked in UBlerns.

Important Dates

- 03/15/2026 Sun 11:59pm – Checkpoint Submission is Due
- 03/29/2026 Sun 11:59pm – Final Submission is Due